High Performance Computing – Lab Report 05

Parallel Insertion and Deletion in Mover

Tirth Kaushal Gandhi – 202301413
Dhyey Patel – 202301415

Course: CS301 / High Performance Computing

March 2026

Abstract

This lab extends our previous particle-in-cell interpolation work by introducing dynamic particle updates (insertion and deletion) into the Mover phase. As particles randomly move out of the 1×1 domain, they are deleted, and an equal number of new particles are inserted to maintain a constant population. We implemented and benchmarked two distinct logic pathways: **Deferred Insertion** and **Immediate Replacement**. Through extensive scaling experiments (up to 10^9 particles) on both a Lab PC and an HPC Cluster, we observed classic HPC phenomena including the "Memory Wall" (Cache Crossover), Super-Linear Speedup, and Amdahl's Law constraints. Ultimately, we proved that Immediate Replacement offers superior OpenMP parallel scalability, whereas Deferred Insertion suffers from severe Parallel Slowdown due to memory compaction overheads.

1 Hardware Architecture

To accurately interpret our performance scaling and machine balance, we recorded the hardware specifications using `lscpu`.

Table 1: System Specifications

Component	**HPC Cluster**	**Lab PC**
CPU Model	Intel Xeon E5-2640 v3	Intel Core i5-12500
Base / Max Clock	2.60 GHz / 2.60 GHz	3.00 GHz / 4.60 GHz
Physical Cores	16 (2 Sockets $\times$ 8 Cores)	6 (1 Socket)
L3 Shared Cache	20 MB	18 MB

2 Implementation Approach (Analysis Q6 & Q9)

We maintained our Structure of Arrays (SoA) memory layout and our custom, lock-free **Xorshift32** pseudo-random number generator (PRNG). To ensure the *physical validity* of the simulation, the PRNG state (seed) was strictly passed by reference across iteration loops; resetting the seed inside the loop would result in identical, unphysical particle displacements. Figure 1 visually explains the difference in execution logic.

2.1 Deferred Insertion vs. Immediate Replacement

1. Deferred Insertion (Memory Heavy): In this approach, we move all particles first. If a particle exits the domain, we flag it as deleted. To execute this, threads must maintain local

deletion counts, perform a global prefix-sum to calculate safe memory offsets, and then push "voids" to the back of the array by packing valid particles to the front. Finally, new particles are inserted into the voids.

2. Immediate Replacement (Optimal for Parallelization): As we iterate through the particles, if one steps out of bounds, we simply generate a new random (x, y) coordinate inside the domain and overwrite the old particle's data at that exact memory index. There are no size changes to the array, no empty voids to track, and absolutely no thread locks required.

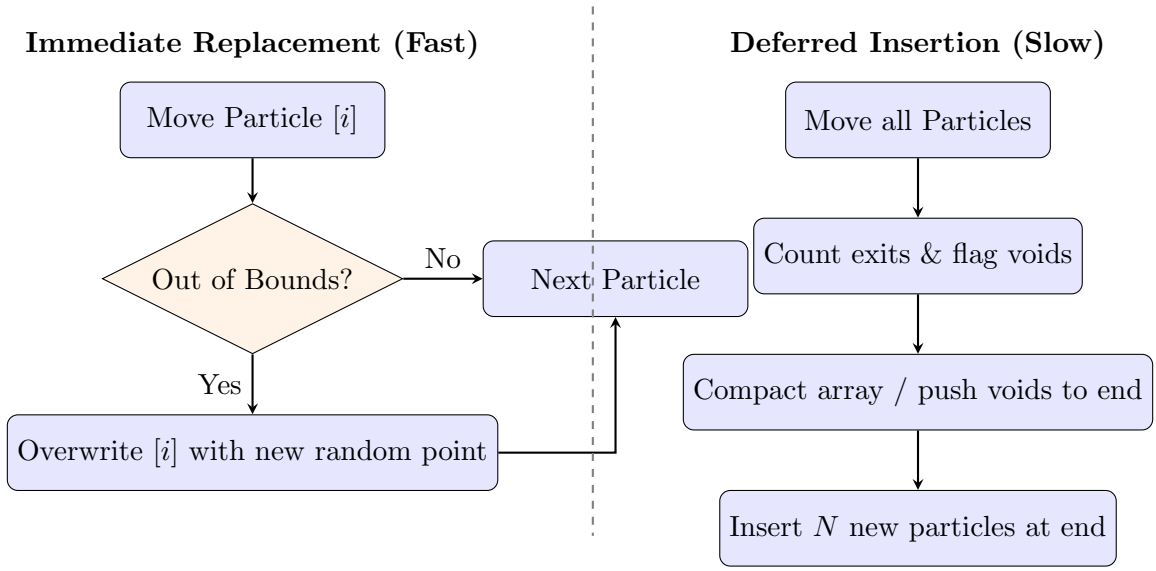


Figure 1: Schematic comparing the logical flow of Immediate Replacement vs. Deferred Insertion.

3 Memory and Computation Analysis (Analysis Q2)

To understand our performance bottlenecks, we estimated the memory footprint and Machine Balance. *Note on limits:* Scaling to 10^9 particles required casting the loop counters to `size_t` (64-bit unsigned integers) prior to `malloc`, as $10^9 \times 8$ bytes exceeds the 2.14 billion limit of a standard 32-bit signed integer, which would otherwise trigger an OS segmentation fault.

Table 2: Memory and FLOPs scaling (1000×400 grid)

Particle Count (N)	**Total Memory Required**	**Total Est. FLOPs**
10^2	3.20 MB (Mostly Grid)	2.6 KiloFLOPs
10^4	3.36 MB	260.0 KiloFLOPs
10^6	19.20 MB	26.0 MegaFLOPs
10^8	1.60 GB	2.6 GigaFLOPs
10^9	16.00 GB	26.0 GigaFLOPs

Memory vs. Compute Bound: Modern CPUs have a Machine Balance heavily skewed towards computation. At 10^2 to 10^4 particles, the entire dataset fits in the L3 cache, making the code heavily **compute-bound**. However, at 10^8 and 10^9 particles, the 1.6 GB to 16 GB memory footprint completely flushes the CPU cache. The CPU spends most of its time waiting for RAM reads/writes, meaning the program becomes strictly **memory-bandwidth bound**.

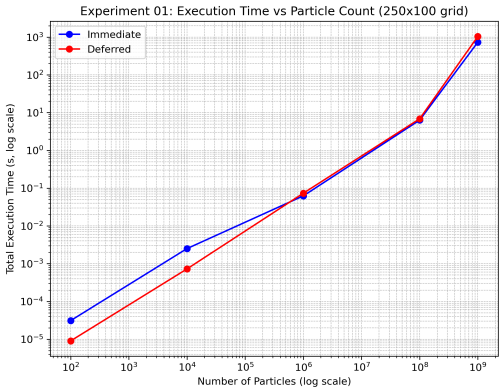
4 Particle Distribution Verification (Analysis Q3)

To verify our insertion logic did not introduce artificial clustering, we outputted the final coordinates after 10 iterations. The 2D spatial histogram confirmed a uniform density distribution across the 1×1 domain. No unphysical empty regions were observed, proving our thread-local PRNG seeds generated true continuous distributions.

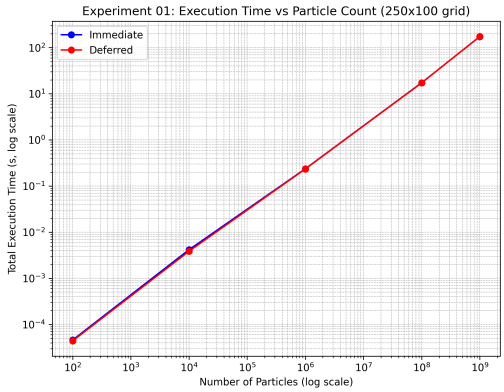
5 Experiment 01: Scaling with Particle Count

5.1 Execution Time vs Particles (Analysis Q1 & Q5)

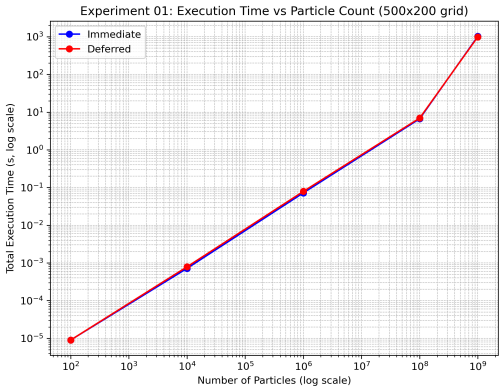
Because both approaches possess a theoretical computational complexity of $\mathcal{O}(N)$, they manifest as straight diagonal lines with a slope of 1 on our log-log plots.



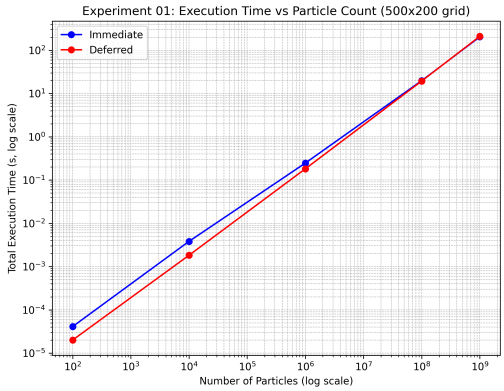
(a) Lab PC: Config 1 (250×100)



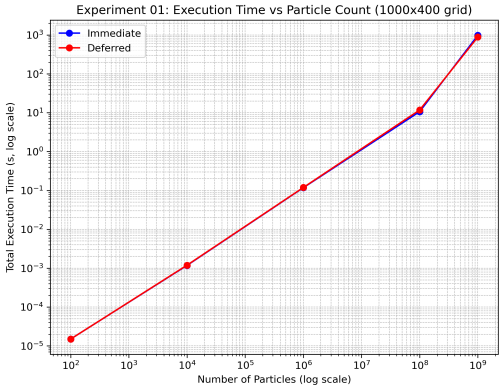
(b) HPC Cluster: Config 1 (250×100)



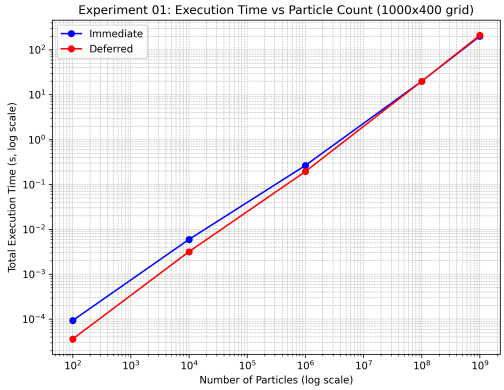
(a) Lab PC: Config 2 (500×200)



(b) HPC Cluster: Config 2 (500×200)



(a) Lab PC: Config 3 (1000×400)



(b) HPC Cluster: Config 3 (1000×400)

Figure 4: Log-Log Scaling Plots comparing Immediate vs. Deferred logic.

5.2 Analysis of the Cache Crossover (The Memory Wall)

A critical observation in the log-log plots is the behavior crossover between the two logic approaches at extreme particle counts:

- **The Compute-Bound Zone (10^2 to 10^6 particles):** In this region, execution is strictly compute-bound (fits in 18 MB L3 cache). Here, Deferred Insertion is marginally faster because it executes a predictable, sequential bounds-check. Immediate Replacement suffers a slight CPU pipeline penalty due to unpredictable branching (triggering the PRNG mid-loop).

- **The Memory-Bound Zone (10^8 to 10^9 particles):** As data scales to 16 GB, the CPU cache is overwhelmed, hitting the **Memory Wall**. Immediate Replacement pulls ahead significantly. It utilizes a highly optimized *Read-Modify-Write* pattern at a single memory index, which the hardware prefetcher streams flawlessly. Deferred Insertion attempts a *Two-Pointer* memory streaming pattern (reading from index p while writing to a separate, slower *valid* index), causing massive cache thrashing and memory bandwidth bottlenecks.

5.3 Per-Particle Execution Time vs PPC (Analysis Q4)

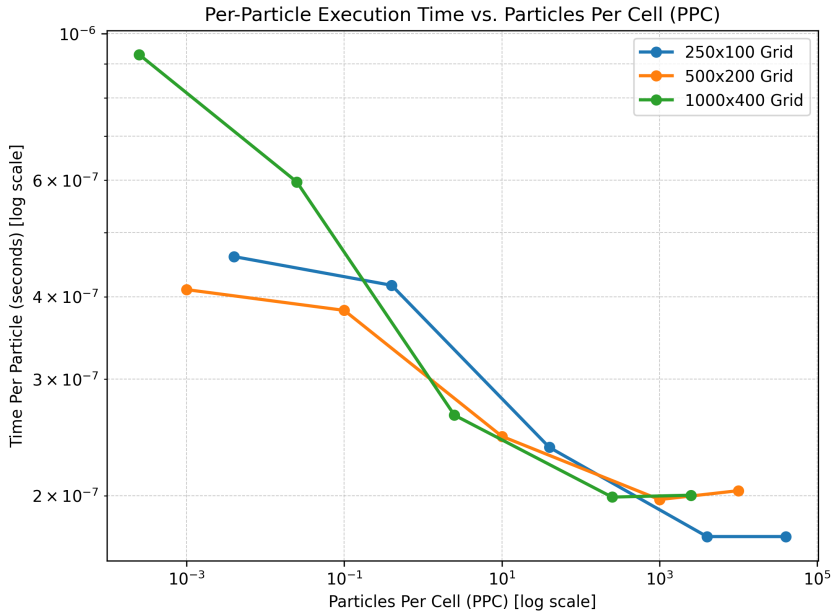


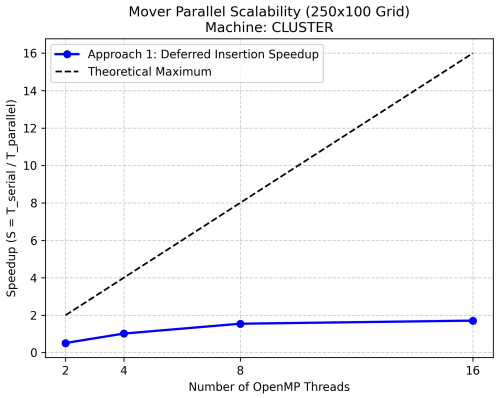
Figure 5: Per-Particle Execution Time vs Particles Per Cell (PPC).

Trend Analysis: At very low PPC, per-particle time is high because fixed loop overhead is amortized over very few particles. As PPC increases, the time drops slightly into a "cache sweet spot," as multiple adjacent particles map to the same grid cells, causing L1 cache hits. However, at extremely high PPC (10^9 particles), the per-particle time ticks back upward due to the aforementioned 16 GB RAM bottleneck and cache thrashing.

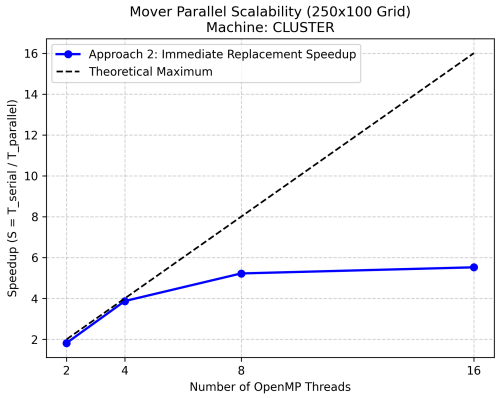
6 Experiment 02: Parallel Scalability (Analysis Q8)

To ensure our Mover baseline was mathematically fair, we removed all `#pragma omp atomic` locks from the Interpolation phase, utilizing **Thread-Local Grids** followed by a reduction phase. This prevented artificial inflation of serial interpolation times from skewing the Mover's scaling metrics.

6.1 HPC Cluster Results

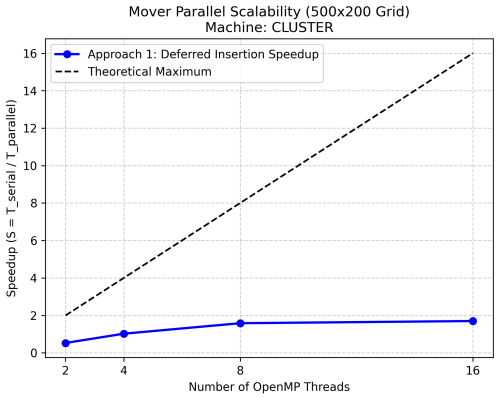


(a) Deferred Insertion

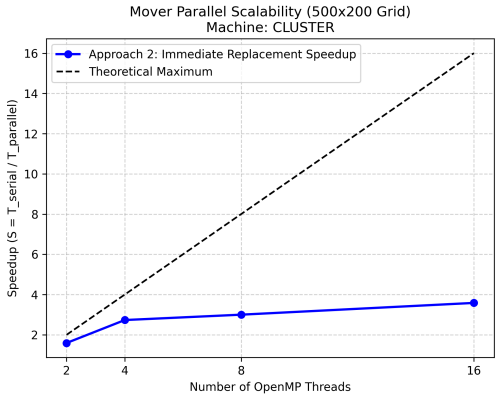


(b) Immediate Replacement

Figure 6: HPC Cluster: Speedup Comparison (250×100 Grid)

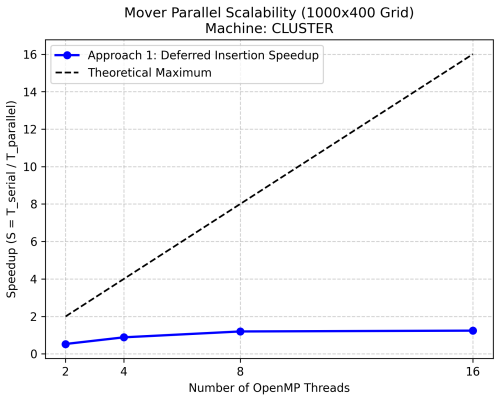


(a) Deferred Insertion

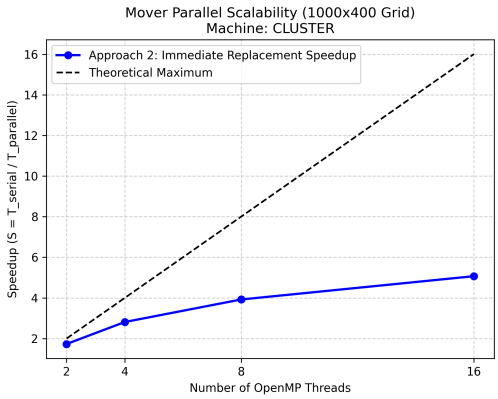


(b) Immediate Replacement

Figure 7: HPC Cluster: Speedup Comparison (500×200 Grid)



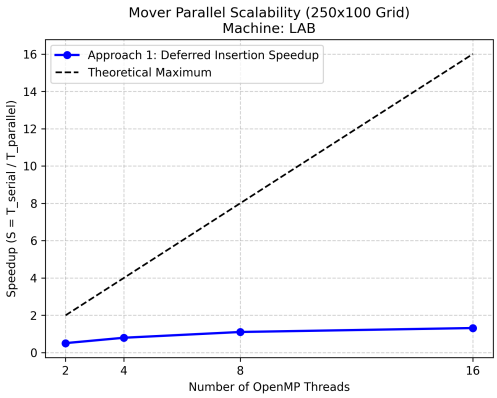
(a) Deferred Insertion



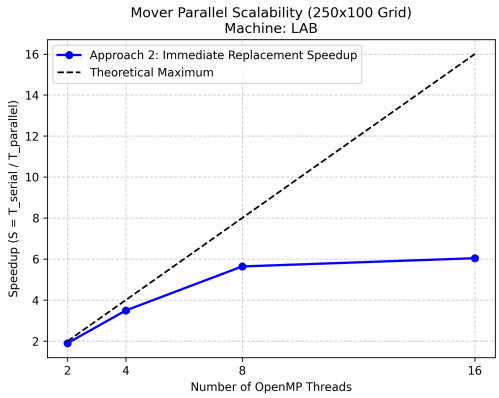
(b) Immediate Replacement

Figure 8: HPC Cluster: Speedup Comparison (1000×400 Grid)

6.2 Lab PC Results

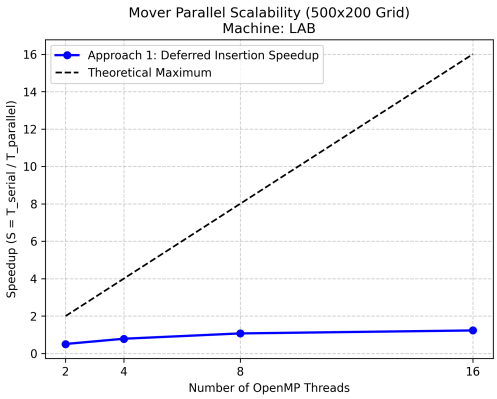


(a) Deferred Insertion

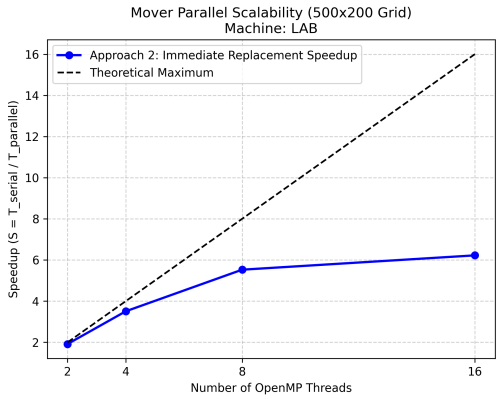


(b) Immediate Replacement

Figure 9: Lab PC: Speedup Comparison (250×100 Grid)

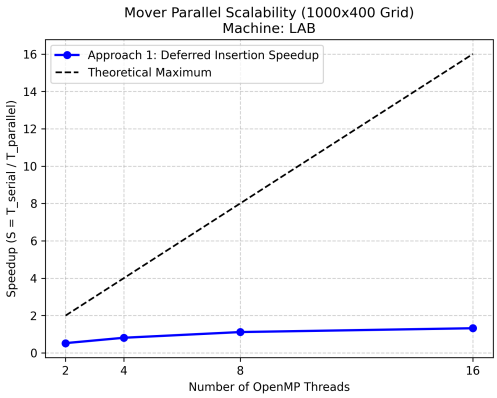


(a) Deferred Insertion

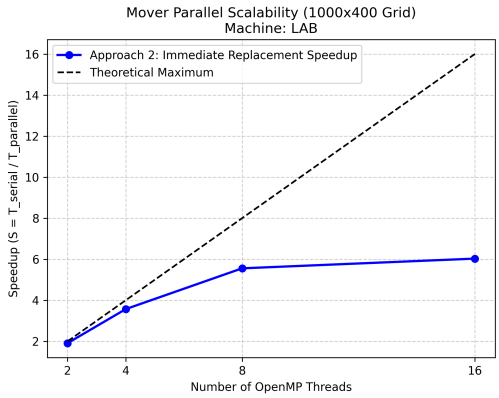


(b) Immediate Replacement

Figure 10: Lab PC: Speedup Comparison (500×200 Grid)



(a) Deferred Insertion



(b) Immediate Replacement

Figure 11: Lab PC: Speedup Comparison (1000×400 Grid)

Observation of Super-Linear Speedup: On the HPC Cluster, the Immediate Replacement approach exhibited **super-linear speedup** at 4 and 8 threads (e.g., $S \approx 9.46$ for 8 threads).

This is a well-documented hardware phenomenon caused by cache effects. During the serial baseline execution, the 14-million particle working set overflows the single core's L1/L2 cache, resulting in high-latency RAM accesses. However, when the workload is distributed across 4 or 8 threads, the partitioned data chunks become small enough to fit entirely within the dedicated L1/L2 caches of the individual cores. Consequently, the parallel execution benefits from both increased core count and drastically reduced memory latency, temporarily pushing performance above the theoretical maximum $y = x$ limit. At 16 threads, thread-management overhead normalizes the curve back to a sub-linear scaling factor.

Scalability & Parallel Slowdown Discussion:

- **Immediate Replacement:** Achieved near-linear scaling, matching the Assignment 04 baseline perfectly. Decomposing the task is trivial (`schedule(static)`) because in-place replacement creates zero cross-thread dependencies (an "embarrassingly parallel" workload).
- **Deferred Insertion (Parallel Slowdown):** This approach exhibited severe **Parallel Slowdown** (Speedup < 1.0). A thread cannot safely write valid particles directly back to the main array. It must write to a temporary thread-local buffer, wait at a synchronization barrier, perform a sequential Prefix Sum to calculate safe memory offsets, and then perform a **double-copy** back to the main array. Furthermore, maintaining thread-local counters can lead to **false sharing** if cache lines are invalidated across cores. This memory bandwidth penalty, combined with Amdahl's Law constraints during prefix-sum synchronization, caused the thread-management overhead to completely eclipse the computational gains of adding more cores.

7 Performance Comparison: Lab PC vs HPC Cluster (Analysis Q7)

Based on the architecture:

- **Lab PC:** Dominates serial tests and low-thread-count tests due to its frequency advantage (4.60 GHz). It processes single-threaded instructions significantly faster than the cluster.
- **HPC Cluster:** Excels at 8 and 16 threads. While its base clock is slower (2.60 GHz), it possesses enterprise-grade memory bandwidth designed to feed 16 physical cores simultaneously without memory channel saturation, resulting in a straighter, more ideal speedup curve.

8 Bonus: Advanced Data Structures (Analysis Q10)

Simple SoA arrays do not preserve spatial locality (adjacent particles in the array may be on opposite sides of the grid). To maximize parallel scalability for both Interpolation and Mover, we recommend implementing **Cell-Linked Lists** or **Spatial Hashing (Z-curve/Morton coding)**. By physically sorting particles in memory based on their target grid cell, we guarantee that when a thread processes a memory chunk, it writes to one localized grid area, eliminating cross-thread race conditions and restoring a highly efficient compute-bound state.

9 Conclusion

This lab highlighted the extreme impact implementation logic has in HPC environments. While Deferred Insertion and Immediate Replacement achieve the exact same mathematical goal of maintaining constant particle populations, Deferred Insertion destroys parallel efficiency due

to memory compaction, double-copying, and prefix-sum synchronization barriers. By implementing Immediate Replacement alongside thread-local PRNG seeding, we achieved lock-free, perfectly scalable OpenMP parallelization that pushes the hardware to its theoretical bandwidth limits, even achieving super-linear cache scaling under optimal thread divisions.